



## **Building a RAG System with DuckDB**

Chandan Dutta – Principal Consultant, Data Services

# Introductions



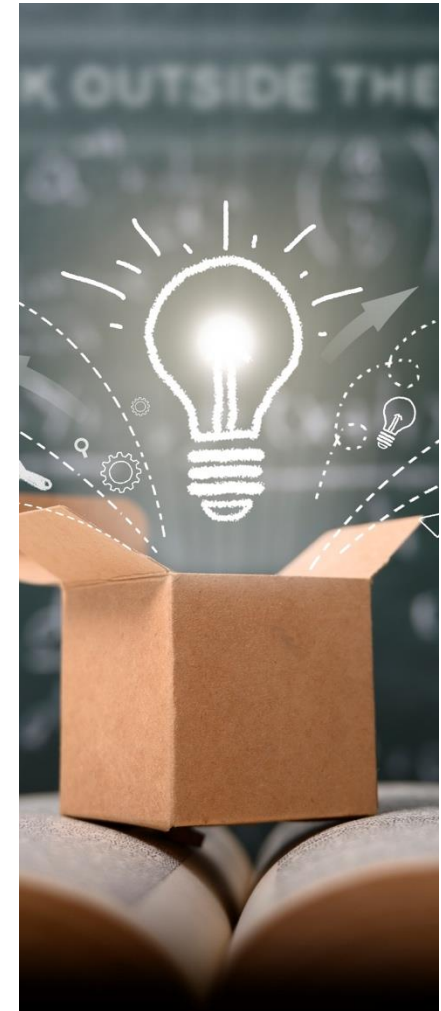
2 years at Improving

20 years of IT experience across diverse industry verticals, including healthcare, banking, and manufacturing

# For discussion today...

- The Problem We're Solving?
- Why DuckDB?
- The Journey Ahead
- What Makes This Practical

“Simplicity is the ultimate sophistication.” — Leonardo da Vinci



## The Problem we are solving

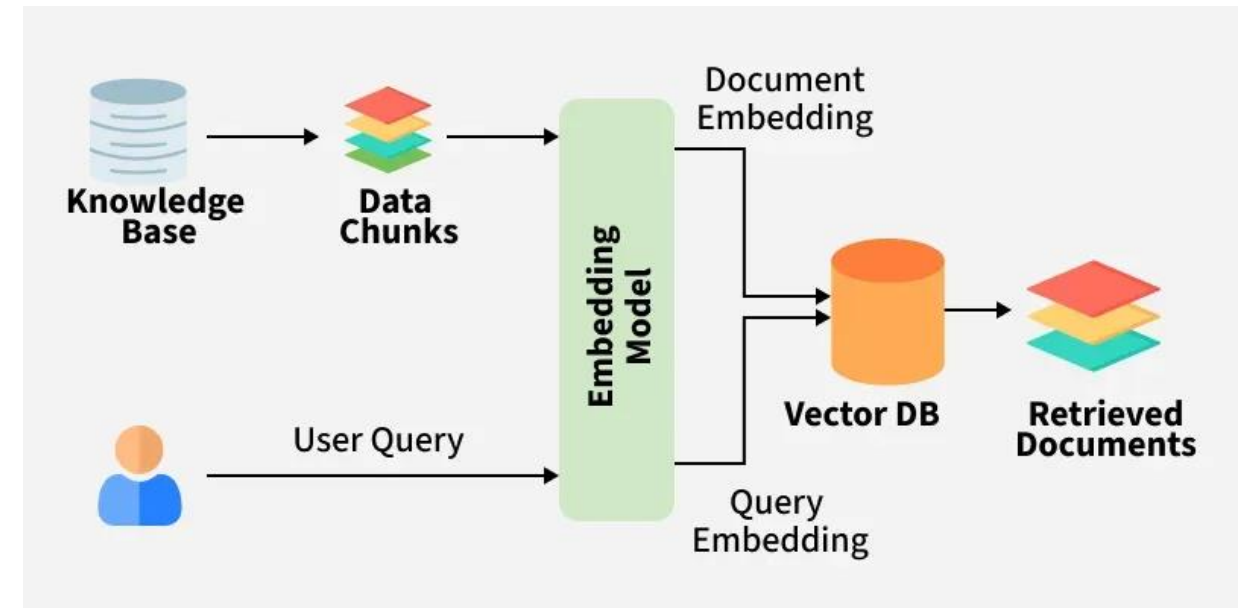
# What is RAG?

## Retrieval-Augmented Generation

A powerful AI architecture that combines search and generation:

🔍 **Retrieval:** Finds relevant information from your own private documents.

🧠 **Generation:** LLM generates answers based strictly on the retrieved context.



## Why DuckDB?

# Why DuckDB for RAG?



## In-Process

No separate server needed.  
Runs directly in your application  
for zero network latency.



## Parquet Support

Native zero-copy columnar  
scanning for blazing fast data  
loads from Parquet files.



## Vector Storage

Built-in support for FLOAT[]  
arrays to store high-dimensional  
embeddings natively.

# System Architecture

## Data Layer

- Parquet Files
- DuckDB Tables
- Chunks Table
- Embeddings Table

## Processing Layer

- Text Chunking
- Embedding Gen
- Similarity Search
- Prompt Builder

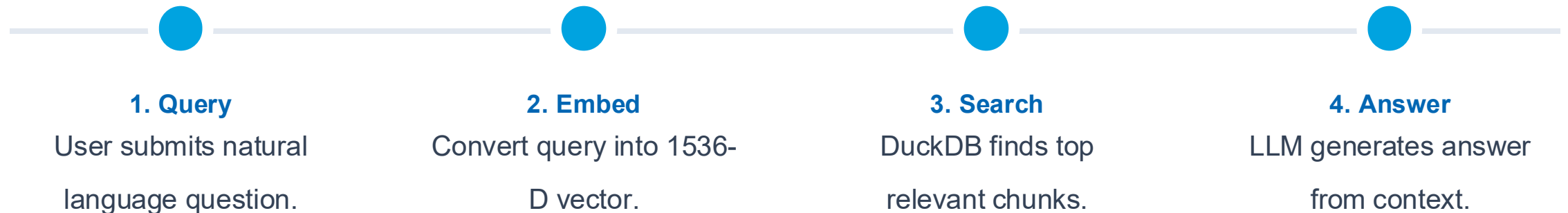
## AI/LLM Layer

- text-embedding-3
- gpt-4o-mini
- Final Answer



## The Journey Ahead

# RAG Query Flow



**Key Insight:** Similarity search converts the semantic relationship between question and document into a mathematical distance calculation.

# System Requirements

| Component       | Minimum                                | Recommended     |
|-----------------|----------------------------------------|-----------------|
| Python          | 3.9+                                   | 3.11+           |
| DuckDB          | 0.9.0+                                 | 1.1.0+ (latest) |
| RAM             | 4 GB                                   | 8 GB+           |
| Python Packages | duckdb, openai, pandas, numpy, pyarrow |                 |

# Step 1: Load Parquet Data

## Native **Columnar** Loading

DuckDB reads Parquet files directly with native columnar scanning - no ETL pipeline needed!

```
CREATE TABLE docs AS  
SELECT * FROM read_parquet('data/*.parquet');
```



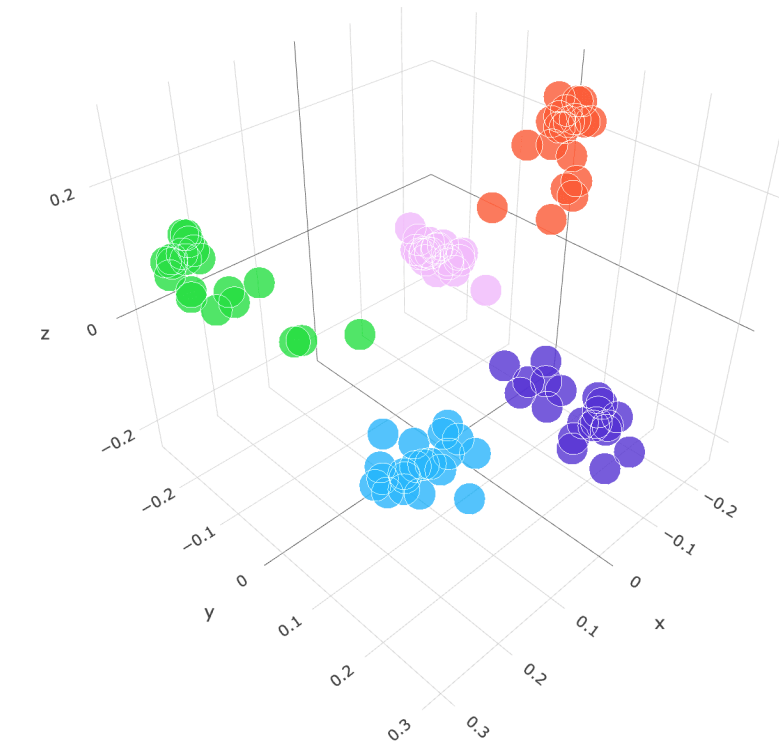
## Step 2: Text Chunking Strategy

| Strategy           | Pros                | Best For                   |
|--------------------|---------------------|----------------------------|
| Fixed-size window  | Simple, predictable | Uniform text corpora       |
| Sentence-based     | Respects grammar    | Articles, prose            |
| Semantic/Recursive | Highest coherence   | Complex, mixed-format docs |

Our Approach: 512-character chunks with 64-character overlap for context preservation.

# Step 3: Generate Embeddings

● animal ● athlete ● film ● transportation ● village



## Capturing Semantic Meaning

Embeddings convert text into 3072-dimensional vectors.

Similar meanings occupy similar space in the vector field.

- ✓ model: text-embedding-3-large
- ✓ Batch Size: 100 chunks per call

# Step 4: Store in DuckDB

## Vector-Enhanced Schema

DuckDB natively supports `FLOAT[]` arrays, making it an excellent persistent vector store.

### Embeddings Table

chunk\_id (UUID)  
embedding (FLOAT[3072])

### Key Benefits

Persistent file storage.  
No separate vector DB needed.  
Embed once, query forever.

# Step 5: Retrieval & Search

## Native Cosine Similarity

Use DuckDB's built-in vector functions for high-speed similarity search within SQL.

```
SELECT list_cosine_similarity(query_vec, embedding) AS score...
```

## Sort by Score

Return Top-K most similar segments.

## Metadata Filter

Use SQL WHERE to filter by section.



# Step 6: Prompt Engineering

## System Message

"Answer ONLY based on the provided context. If the answer is not present, say 'I don't have that information'."

## User Message

Combines the retrieved chunks, metadata (for citations), and the original question into one focused prompt.

# LLM Parameters

| Parameter   | Value       | Rationale                                  |
|-------------|-------------|--------------------------------------------|
| Model       | gpt-4o-mini | Optimal price/performance for RAG          |
| Temperature | 0.2         | Low creativity ensures factual consistency |
| Max Tokens  | 1024        | Balance between detail and cost            |



# Key Takeaways



## Simple

No server needed -  
DuckDB runs in-  
process.



## Fast

Native Parquet and  
columnar  
processing.



## Cost-Effective

Cache embeddings  
once, query many  
times.

**DuckDB + RAG = Production-Ready AI in Minutes**

improving 

**Thank You!**

# GitHub Repository

[https://github.com/](https://github.com/Chandandutta/RAGonDuckDB) Chandandutta / RAGonDuckDB

Chandan.Dutta@Improving.com

# Continuing The Conversation



Connect with me:

Email: [Chandan.Dutta@improving.com](mailto:Chandan.Dutta@improving.com)

LinkedIn: <https://www.linkedin.com/in/chandandutta/>

Connect with Improving:

<https://improving.com/contact>

**Additional Material  
(On Demand)**



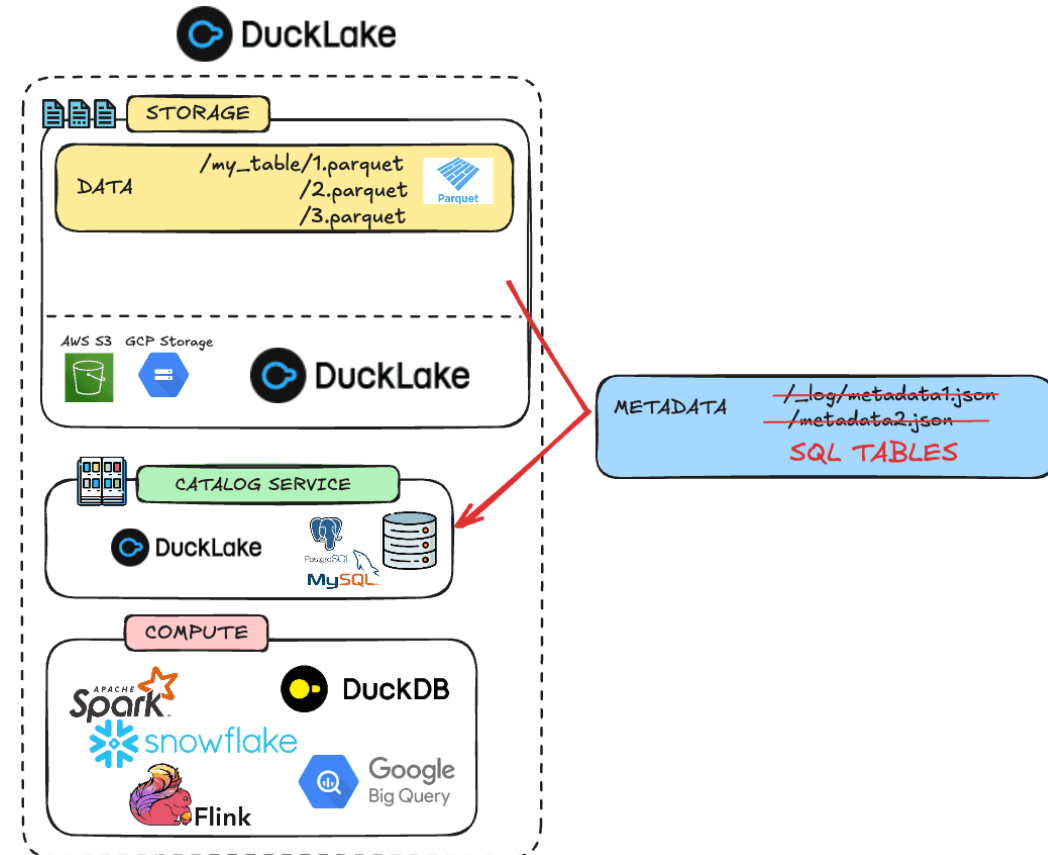
# DuckLake: Lakehouse Storage for DuckDB

## What is DuckLake?

- DuckLake is a Lakehouse storage layer built on top of DuckDB
- Combines the performance of DuckDB with the scalability of data lakes
- Supports Delta Lake, Iceberg, and Hudi table formats
- Enables ACID transactions on cloud object storage (S3, GCS, Azure Blob)

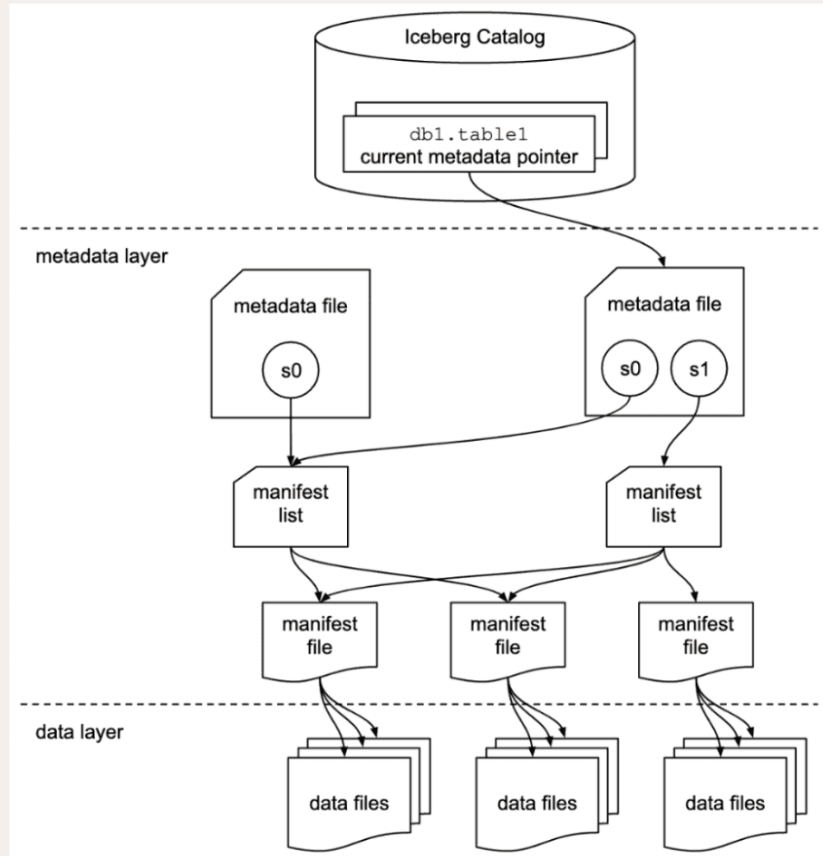
## Key Features:

- Zero-copy integration with DuckDB's query engine
- Time travel and versioning capabilities
- Schema evolution and enforcement
- Efficient metadata management for large datasets

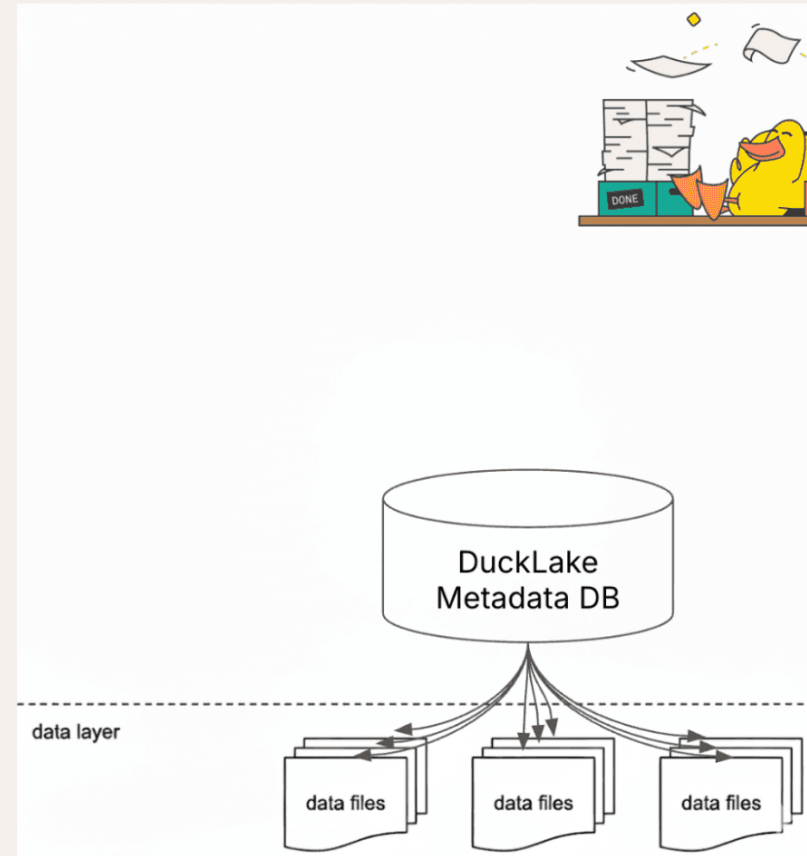


# Accessing Data File

## Apache Iceberg



## DuckLake



# Quack Protocol: DuckDB-to-DuckDB Communication

## What is Quack Protocol?

- Native protocol for DuckDB instance communication
- Enables distributed query execution across DuckDB nodes
- Optimized for columnar data transfer and zero-copy operation
- Built on Apache Arrow Flight for efficient data exchange

## Key Capabilities:

- Remote query execution on distributed DuckDB instances
- Efficient data shuffling for distributed joins and aggregations
- Authentication and encryption support
- Low-latency communication for real-time analytics
- Automatic query plan distribution and coordination

**QUACK: A CLIENT-SERVER  
PROTOCOL FOR DUCKDB**

IF IT QUACKS  
LIKE A DUCK...



WE <3 DUCKDB WE <3 DUCKDB WE <3 DUCKDB WE <3 DUCKDB WE <3 DUCKDB